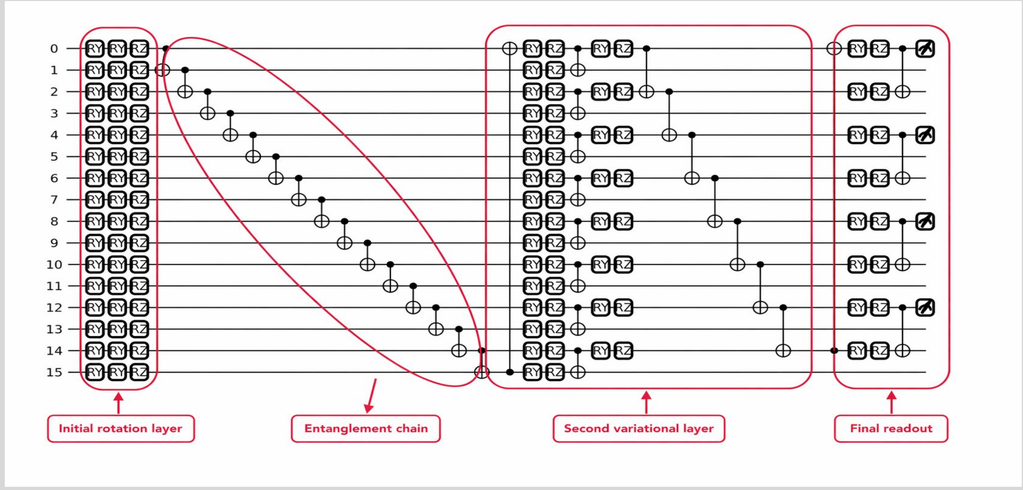
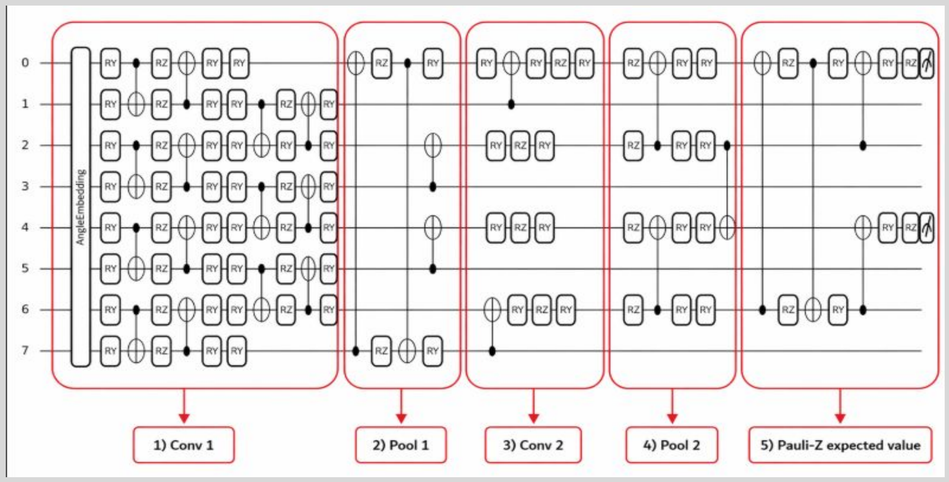
Building Student-Friendly Quantum Machine Learning at CAIR: How Can QML Be Made More Accessible to Students?

Simran (Summer) Malik, Timothy Lin, Jonathan De La Paz Posadas
Charlotte AI Research Lab

Introduction

Quantum machine learning is often described as the future, but for many students, it still feels hard to access. At Charlotte AI Research, I started the Quantum Machine Learning division to create a space where students could explore QML through real projects and peer learning. Timothy and Jonathan have made major contributions by building projects that make hybrid quantum-classical methods more approachable for other students, while also helping other students think through and improve their own models. Together, this work explores one central question: **how can quantum machine learning be made more accessible to students?**





Experimental Workflow

- Start from machine learning tasks that students can realistically understand and test
- Build hybrid quantum-classical workflows using familiar Python-based tools
- Compare circuit and model architectures against classical baselines
- Document patterns, tradeoffs, and performance differences across experiments
- Ask how QML can become both more accessible to build with and more competitive with classical approaches

One architecture is more hierarchical and compression-driven, while the other is more variational and layer-driven. This highlights two different ways hybrid quantum models can organize trainable structure and feature transformation.

Approaches

A central insight of this work is that hybrid quantum-classical learning does not appear to follow a single universally optimal architecture. Instead, these projects suggest that circuit structure, representation strategy, and task domain interact in ways that meaningfully shape model behavior. One architecture emphasizes hierarchical compression, while another relies on a more variational, layer-wise transformation of features, indicating that hybrid QML may be better understood as a family of task-dependent design strategies rather than a single model paradigm. In that sense, this work points toward an important practical conclusion for student-facing QML: one size does not fit all

Quantum-Enhanced Representation Learning in NLP
This workflow positions NLP as a non-visual benchmark for hybrid quantum-classical learning, probing whether quantum-enhanced representations remain informative in high-dimensional language space.

Hybrid NLP Model

```
class HybridSentimentModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, n_qubits):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim)      ← Embedding layer
        self.pre_q = nn.Linear(embed_dim, n_qubits)               ← Classical projection
        self.q_layer = qml.qnn.TorchLayer(circuit, weight_shapes) ← Quantum layer
        self.post_q = nn.Linear(n_qubits, 1)                      ← Output layer

    def forward(self, x):
        x = self.embedding(x)      # embedding
        x = x.mean(dim=1)          # pooling
        x = self.pre_q(x)          # projection
        x = self.q_layer(x)        # quantum
        x = self.post_q(x)         # output
        return x
```

Conclusions

Taken together, these projects suggest that making QML more accessible is not simply a matter of simplifying explanation, but of building better pathways into a genuinely diverse model space. If hybrid quantum learning is task-sensitive, then student access depends on having workflows that make architectural variation easier to build, test, and compare. This is part of the motivation for QMLTorch: a Python-based framework intended to make hybrid quantum-classical experimentation more modular, interpretable, and adaptable across problem settings. Rather than forcing all tasks into one architecture, the long-term goal is to make it easier to discover which hybrid designs are actually worth using, and why.